



UCL ONE

One trusted digital hub for campus information and services

DevDash'26 • Final Technical Report & Evidence Pack

REPORT PURPOSE

This report documents the verified design, architecture, implementation, testing, trade-offs, limitations and evidence for UCL ONE. It is intentionally conservative: claims are only made where the available repository/test/audit evidence supports them.

Team	Project	Submission scope
Maleesha Dewshan Helika Athukoralla Vinuli Ranasinghe	UCL ONE Campus information + services platform	DevDash'26 Code Review 60% + Product Pitch 40%

Prepared from the official DevDash'26 source documents and the latest available project audit artifacts.

Document Control

Item	Value
Project	UCL ONE
Team	Dewshan's Kids
Team members	Maleesha Dewshan • Helika Athukoralla • Vinuli Ranasinghe
Repository	DevDash-26/Dewshan-s-kids
Report purpose	Final technical evidence report aligned to DevDash'26 judging criteria
Evidence state	Latest available audit artifacts and final test-gate records
Important status note	A headline count of "29/33 complete" appears in some project artifacts, while the detailed latest traceability enumerates BR1–BR23, BR25–BR26 and BR28–BR32 as complete (30 items) and BR24, BR27 and BR33 as partial. This report treats the detailed requirement rows as authoritative and flags the headline count discrepancy for final team verification.

Source Basis

The official problem statement defines the business requirements, non-functional requirements, assumptions, constraints and expected deliverables. The official judging criteria allocate 60% to Code Review and 40% to Product Pitch. The rules define the six-hour development window, GitHub submission, commit cadence, disclosure obligations and AI/public-library conditions. [filecite:turn32file9](#) markers are provided in the accompanying chat response; this document uses human-readable source references.

Source ID	Source	Use
S1	DevDash Hackathon Problem Statement.pdf	Requirements, NFRs, assumptions, constraints, deliverables
S2	Judging Criteria (1)(1).docx	100-mark evaluation model and excellence descriptors
S3	DEV DASH '26 RULES AND REGULATIONS (Finalized as of 17th September).docx	Hackathon rules, conduct, submission and timing
E1	Latest repository/audit artifacts available in the project context	Implementation, test, Git and security evidence

1. Executive Summary

UCL ONE addresses the central DevDash'26 problem: students currently rely on disconnected channels such as WhatsApp groups, lecturers, notice boards, societies and word of mouth, with no single channel treated as authoritative. The official problem asks teams to provide one reliable and trusted channel for campus-related information and services; it does not prescribe a particular technology or product shape. [filecite:turn32file9](#)

The implemented solution uses one coherent web experience with three architectural responsibilities: DISCOVER campus information, ACT through transactional services, and ASSIST through an information-grounded assistant. A reusable Strapi content engine covers the large information surface; Firestore handles user-specific transactional workflows; Firebase Authentication provides identity; Firestore Security Rules provide the server-side authorization boundary; and the AI layer combines content retrieval with a Gemini integration and a deterministic fallback. [filecite:turn35file0](#)

The latest final test-gate evidence records 29 pure-logic unit tests, 44 Firestore rules/booking-integration tests, 3 authentication tests, a committed Playwright golden-path suite with 12 steps and zero console errors, a 5/5 CMS security verification script, clean TypeScript checking, successful linting with warnings but no errors, and a successful production build with route-level code splitting. This is 76 automated tests in total, plus the E2E suite and security script.

The engineering story is therefore not "everything was built." It is: the team prioritised a high-coverage, reusable architecture; built the highest-value transactional workflows with server-side authorization; validated the core path with layered tests; fixed defects discovered by testing; and explicitly documented what remains incomplete.

2. Problem Definition & Business Context

The problem statement describes a structural information-fragmentation issue. Campus information is spread across informal and disconnected channels, affecting announcements, events, lost property, campus services and society activity. The absence of a shared authoritative channel also causes administrative staff to become a fallback source for routine questions. [filecite?turn32file9](#)

2.1 Project objectives

- Provide one coherent access point for campus information and services.
- Improve visibility of events, societies and campus services.
- Reduce the effort required to find routine campus information.
- Provide a foundation that can be extended beyond the hackathon.

2.2 Constraints that shaped engineering decisions

- The solution had to be realistic for a university environment and its operational capacity.
- The university did not prescribe a technology stack or vendor.
- The prototype had to be built within the six-hour hackathon window.
- The design needed to respect student/staff privacy and authorised access.
- The problem statement explicitly allowed teams to focus on a subset rather than treating all categories equally.

[filecite?turn32file0](#)

3. User Analysis

User	Primary needs	Representative requirements
Students	Find relevant information; complete common campus actions without knowing informal processes.	BR1, BR2, BR3, BR4, BR6, BR7, BR8, BR9, BR10, BR17, BR21, BR33
Staff	Publish/maintain official content; triage student requests; manage operational workflows.	BR11, BR12, BR17, BR21, BR8
Administrators	Manage higher-privilege reference data and operational access; approve requests.	BR8, BR11, BR12
Future maintainers	Understand, extend and sustain the system after the hackathon.	NFR5

4. Requirement Strategy & Prioritisation

The official judging model makes requirement coverage the largest Code Review component at 45 marks, while Product Pitch also evaluates completion, understanding and prioritisation. The team therefore prioritised a reliable core loop before expanding breadth. [filecite?turn36file6](#)

Level	Priority	Examples	Reason
Level 1	Essential reliable functionality	Authentication, RBAC, dashboard, content discovery, booking, lost & found	Protect the demonstrable core and high-value workflows.
Level 2	Meaningful technical depth	Targeting, conflict detection, department permissions, validation, grounded retrieval, failure states	Turn basic pages into defensible engineering.
Level 3	Coverage & differentiation	22-category content engine, AI integration/fallback, reusable patterns	Expand requirement coverage without multiplying infrastructure.
Polish	Validation & evidence	E2E smoke test, security verification, docs, demo readiness	Convert working code into verifiable submission evidence.

A key design decision was to avoid implementing every information requirement as a bespoke mini-application. The latest audit describes a reusable content engine covering 22 categories, allowing information-only requirements to share a schema and UI pattern while engineering time was concentrated on bespoke workflows such as booking and user actions. [filecite?turn37file3](#)

5. Solution Overview

Layer	Responsibility	Representative implementation
DISCOVER	Campus information discovery	Strapi content engine, Search & Discover, dashboard, targeting
ACT	User-specific transactional services	Firestore collections and workflows for bookings, interests, lost & found, support, feedback, facility issues
ASSIST	Natural-language information access	Retrieval + Gemini integration + deterministic fallback

The product is deliberately a coherent platform rather than a collection of unrelated mini-apps. The content engine provides a common category model; action workflows provide stronger domain-specific behaviour where state and permissions matter; and the assistant provides a second access path to the same information corpus. [\[filecite\]turn35file0](#)

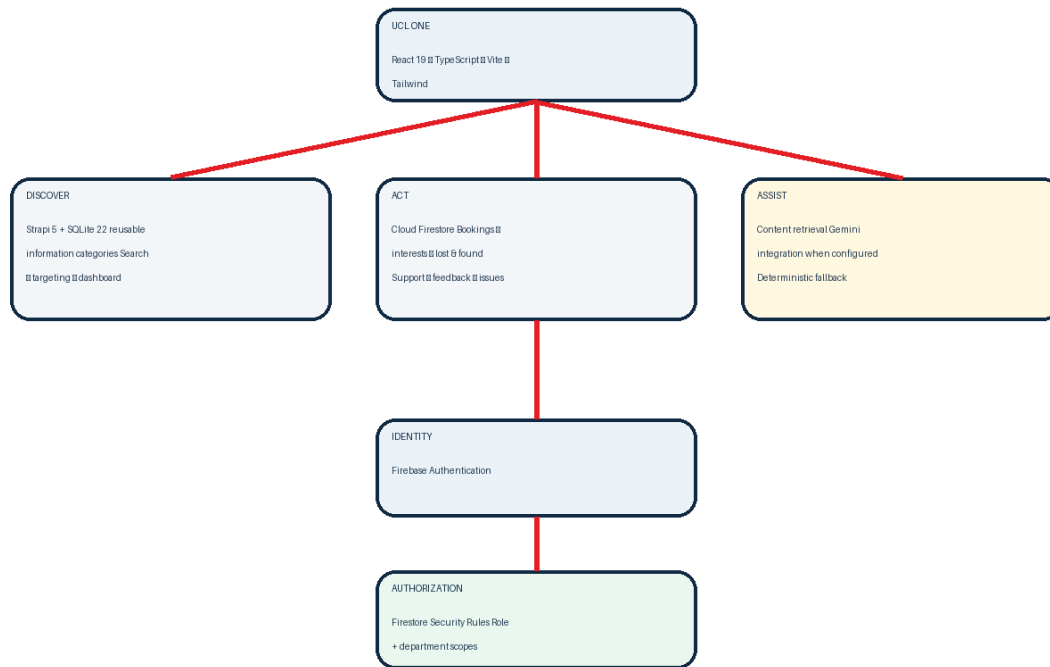


Figure 1. Verified UCL ONE logical architecture.

6.1 Frontend

The frontend is a React 19 + TypeScript application built with Vite and styled with Tailwind CSS. The project uses route-based pages and a shared UI vocabulary (cards, buttons, badges, empty/error states). The latest audit also records route-level code splitting, producing approximately 20 page-specific chunks while retaining a larger vendor chunk dominated by the Firebase SDK. [\[filecite\]turn34file7](#)

6.2 Content layer

Strapi Community Edition provides the reusable content engine. A single content-item model is differentiated by a category enum, covering the information categories required by the problem statement. This reduces the marginal implementation cost of adding information categories and gives staff an administrative publishing workflow. [\[filecite\]turn29file4](#)

6.3 Transaction layer

Cloud Firestore stores user profiles, room bookings, event/society interest, lost & found records, academic support requests, feedback and facility issues. These are transactional or user-specific concerns where per-document access rules matter.

6.4 Authentication and authorization

Firebase Authentication provides identity. Firestore Security Rules provide the actual server-side authorization boundary. The implementation distinguishes STUDENT, STAFF and ADMIN and includes staff-department scopes. This is materially stronger than relying only on client-side route hiding. The latest test evidence includes 44 rules/integration tests against a live Firestore emulator. [filecite?turn37file2](#)

6.5 No custom API server

The latest forensic audit confirms there is no conventional custom REST/API backend between the frontend and the two data systems. The frontend uses typed client libraries to communicate with Firestore and Strapi directly. This is a deliberate simplification appropriate to the six-hour constraint, but it also explains why production concerns such as secret-proxying and centralised transaction enforcement remain future work. [filecite?turn34file4](#)

7. Data Architecture & Core Workflows

Workflow	Data source	Core logic	Evidence
Authentication	Firebase Auth + users profile	Email/password auth; profile contains role and targeting attributes	3/3 Auth tests; E2E login
Targeting	Strapi content item	Audience/faculty/programme/yearGroup filtering and ranking	29 unit tests include BR2 targeting
Events	Strapi + Firestore	EVENT content + deterministic event-interest document	E2E event-interest step; unit/rules tests
Societies	Strapi + Firestore	SOCIETY content + one interest record per student/society	E2E society sign-up
Room booking	Firestore	Availability query + overlap detection + request/approval state	44 rules/integration tests; E2E booking/approval
Lost & Found	Firestore	Typed report, browse/filter, owner/staff resolution	E2E lost & found
Academic support	Firestore	Typed request + status tracking + staff triage	Route + Firestore implementation evidence
Feedback	Firestore	Student subject/message + staff response/status	Route + rules implementation
Facility issues	Firestore	Typed issue report + staff status workflow	Route + rules implementation
AI assistant	Strapi retrieval + Gemini integration/fallback	Whole-word/category-aware retrieval → Gemini if configured → deterministic fallback	29 unit tests include fallback/regressions; live Gemini not demonstrated

8. Authentication, RBAC & Department Permissions

The authorization model is designed around the principle that UI visibility is not the security boundary. The frontend can hide routes or controls for usability, but Firestore Security Rules enforce access on the database boundary.

- STUDENT accounts can access student-facing information and submit permitted transactional records.
- STAFF accounts can manage operational workflows according to role/department scope.
- ADMIN accounts receive the highest administrative permissions.
- Department scope is represented in the user profile and permission mapping, supporting differentiated administrative workflows.
- Privilege escalation and role/department boundaries are exercised by the Firestore rules test suite.

9. Content Management & Targeting

The reusable content model is the core coverage strategy. Instead of building separate storage and page logic for announcements, academic calendar, jobs, wellbeing, library, dining and other information categories, the platform uses a shared content-item model with category and targeting metadata.

Capability	Implementation detail	Engineering value
Targeting	Audience + faculty + programme + yearGroup fields; targeting logic filters/ranks relevant content	One targeting primitive works across many categories
Emergency communication	isEmergency flag pins urgent content and bypasses ordinary targeting	Urgent information is not accidentally hidden from a user segment
Search	Unified content search with category-aware retrieval	One discovery path covers the information surface
Maintenance	Strapi admin auth/RBAC for informational content; staff console for transactional data	Separates content maintenance from student actions

10. Detailed Feature Implementation

Feature	Implementation	Depth assessment
BR1 Unified Access	One Search & Discover surface plus dashboard across 22 content categories.	Strong depth: reusable content architecture rather than 22 independent modules.
BR2 Targeted Announcements	Audience/faculty/programme/yearGroup metadata and filtering.	Strong depth: targeting is tested as filtering, not merely sorting.
BR3/BR4 Events	Event content plus deterministic interest registration and live interest count.	Meaningful depth through idempotent interest records.
BR5/BR6 Societies	Society content plus deterministic student interest records.	Duplicate-safe sign-up model.
BR7 Lost & Found	Typed lost/found reports, browsing, filtering and owner/staff resolution.	Transactional workflow with access control.
BR8 Classroom Booking	Availability check, overlap detection, request state, staff approval/rejection.	High-value technical workflow; approval-time conflict checking is transaction-aware, with a narrow remaining race window.
BR9 Academic Support	Typed study-group/tutoring/mentorship request and staff status workflow.	Actionable support rather than information-only content.
BR11 Content Maintenance	Strapi administrative workflow plus staff console.	Uses a specialised CMS instead of a custom content backend.
BR12 Access Levels	Server-side STUDENT/STAFF/ADMIN rules plus department scopes.	High-value security/authorization depth.
BR17/BR21 Feedback & Issues	Student submission, staff response/status workflow.	Turns communication into trackable actions.

11. AI Assistant, Accuracy-First Design

BR33 carries 9 marks, making it the highest-weighted individual business requirement. The assistant therefore receives separate treatment in this report.

11.1 Intended flow

User question → retrieve relevant UCL ONE content → whole-word/category-aware relevance scoring → supply retrieved content as grounding context → Gemini path when configured → otherwise deterministic fallback → show response and source context.

11.2 What is verified

- The retrieval and ranking layer exists and has regression coverage.
- Whole-word matching fixed substring errors such as “book” matching inside “textbook” and “week” matching inside “weekly”.
- The Gemini prompt separates the user question from system grounding instructions, reducing—but not eliminating—prompt-injection risk.
- The fallback is deterministic, grounded in platform content and clearly labels the no-key path.
- The Gemini model call itself has never executed in the tested environment because the API key remained empty. Therefore live Gemini behaviour is not claimed.

12. Security & Privacy

The security model is layered: Firebase Authentication identifies the caller; Firestore Security Rules enforce transactional access; Strapi provides its own administrative authentication and publishing permissions; the frontend provides usability-oriented role gating.

Control	Verified evidence	Remaining consideration
Authentication	3/3 live Auth-emulator tests: valid login, wrong password, unregistered email	Evaluation environment is emulator-based; production identity infrastructure is not demonstrated.
Firestore authorization	44 rules/integration tests; role, department and privilege-escalation scenarios	roomBookings read access is broader than ideal for signed-in users due to Firestore query constraints.
CMS publishing	5/5 CMS security verification checks pass; anonymous writes blocked; draft-read gap fixed	Production deployment configuration is outside the demonstrated environment.
Secrets	No secrets committed; environment values kept out of source control	A live Gemini key would be better handled behind a server-side proxy.
Input robustness	Required-field validation, inert script-tag behaviour and explicit error handling verified	Formal security penetration testing was not performed.

The earlier audit found a high-severity draft-content exposure in the public Strapi API. The team fixed the issue and the final security verification records 5/5 checks passing. This is an important example of evidence-driven engineering: the security claim is supported by a regression check rather than only by code inspection. [filecite?turn34file7?](#)

13. Non-Functional Requirements

ID	Requirement	Marks	Status	Evidence	Limitation
NFR1	Usability	2	PARTIAL	Consistent navigation, quick-action dashboard, mobile smoke testing	No formal first-time-user usability study; mobile navigation is functional but unconventional.
NFR2	Performance & Scalability	3	PARTIAL	Route-level code splitting; server-side filtered queries; fresh build evidence	Vendor JS remains ~823 kB gzipped/~249 kB; no client-side cache; production-scale load testing not demonstrated.
NFR3	Reliability & Availability	1	PARTIAL	Explicit CMS-unavailable UI state; E2E smoke; successful build/test gate	No production redundancy or failure-domain testing; Firebase/Strapi production deployment not demonstrated.
NFR4	Security & Privacy	2	COMPLETE	44+ rules/integration tests; 5/5 CMS security checks; draft-content exposure fixed; RBAC/department tests	Disclosed trade-offs remain: broader roomBooking reads and client-side Gemini key risk if a key were ever configured.
NFR5	Maintainability	4	COMPLETE	TypeScript clean; lint no errors; shared structure; unused dependencies removed; reproducible E2E suite	Hackathon-sized codebase; longer-term operational maintenance has not been measured.
NFR6	Robustness	3	COMPLETE	Form validation; empty/gibberish AI handling; inert script input; profile-null action failures fixed	No exhaustive fault-injection campaign.

The official NFRs require usability, performance/scalability, reliability/availability, security/privacy, maintainability and robustness. The latest detailed audit classifies NFR4–NFR6 as complete and NFR1–NFR3 as partial.

[filecite?turn34file7?turn32file0?](#)

14. Testing Strategy & Results

Testing was treated as a layered evidence system rather than a single test command. The final gate covers pure logic, database authorization and transactional integration, authentication, end-to-end browser behaviour, security verification, linting, type checking and production build.

Layer	Command	Count	Result	What it proves
Pure logic / unit	npm test	29	PASS	RBAC/department permission map, BR2 targeting, BR8 overlap logic, AI fallback/retrieval regressions
Firestore rules + integration	npm run test:rules	44	PASS	Real emulator authorization; role/department boundaries; booking conflict cases
Authentication	npm run test:auth	3	PASS	Real Auth emulator: valid login, wrong password,

Layer	Command	Count	Result	What it proves
				unregistered email
E2E smoke	npm run test:e2e	1 suite / 12 steps	PASS	Golden path from login through search, events, society, booking, lost & found, AI fallback, logout and staff approval; 0 console errors
CMS security verification	npm run verify:security	5 checks	PASS	Public write/draft-read security regression checks
TypeScript	npx tsc --noEmit / tsc -b	—	PASS	No type errors
Lint	npm run lint	—	PASS	Exit 0; 0 errors; two pre-existing warnings noted
Production build	npm run build	—	PASS	Build succeeded; route-level chunks confirmed

The latest test-gate record totals 76 automated tests: 29 unit + 44 rules/integration + 3 authentication, plus one committed E2E suite and one five-check security script. [filecite?turn36file2](#)

14.1 Defects discovered by testing

- Firestore room-availability query failed because the original rule shape was incompatible with an unfiltered list query. The rule/query contract was corrected and the rules suite re-run.
- AI fallback relevance ranking surfaced a cafeteria menu above events for an “events this week” query. Category-aware ranking was improved and regression tests added.
- Cross-project emulator identity mismatch caused profile documents to be created under an unexpected UID. Emulator configuration and seed behaviour were corrected.
- Several action handlers silently no-op'd when the user profile was unavailable. These were changed to show an explicit message or disable the action.
- The committed Playwright suite initially exposed a repeated-run strict-mode failure caused by duplicate seeded test items. The locator was made deterministic with an explicit first-match rule and the suite re-run successfully.

[filecite?turn33file6?turn36file2](#)

15. Business Requirement Traceability Matrix

ID	Requirement	Official marks	Implementation	UI	Data	Status	Evidence / limitation
BR1	Unified Access	5	Unified content engine, dashboard, Search & Discover	/	/search	COMPLETE	22-category content engine and single discovery path.
BR2	Targeted Announcements	3	Audience/faculty/programme/yearGroup targeting	/	/search	COMPLETE	Targeting filters relevant content.
BR3	Event Visibility	3	EVENT content category + event page	/events	/search	COMPLETE	University/student-organised event visibility.
BR4	Event Interest	1	eventInterests deterministic record + count	/events	Firestore	COMPLETE	Idempotent interest registration.
BR5	Society Visibility	3	SOCIETY content category	/societies	/search	COMPLETE	Society activity visibility.
BR6	Society Sign-up	2	societyInterests deterministic record	/societies	Firestore	COMPLETE	Duplicate-safe sign-up.
BR7	Lost & Found	3	Typed reports, search/filter, resolve	/lost-found	Firestore	COMPLETE	Owner/staff workflow.
BR8	Classroom Booking	5	Availability + overlap detection + request + approval	/rooms	/staff	COMPLETE	High-depth workflow; narrow concurrency limitation disclosed.
BR9	Academic Support	3	Typed request + status + staff triage	/academic-support	Firestore	COMPLETE	Study group/tutoring/mentorship.
BR10	FAQ Access	2	FAQ content category	/search	Strapi	COMPLETE	Unified search.
BR11	Content Maintenance	4	Strapi admin + staff console	/staff	cms/admin	COMPLETE	Authorised maintenance workflows.
BR12	Access Levels	6	STUDENT/STAFF/ADMIN + department scopes	/staff	Firestore Rules	COMPLETE	Server-side role/department enforcement.
BR13	Academic Calendar	3	ACADEMIC_CALENDAR content	/	/search	COMPLETE	Dashboard/date-sorted visibility.
BR14	Student Onboarding	2	ONBOARDING content + yearGroup targeting	/search	Strapi	COMPLETE	Targeted onboarding content.
BR15	Emergency Communication	3	isEmergency pinned dashboard content	/	Strapi	COMPLETE	Emergency content bypasses normal targeting.
BR16	Schedule Changes	1	SCHEDULE_CHANGE content + emergency flag	/	/search	COMPLETE	Closure/change visibility.
BR17	Feedback Loop	1	Feedback submission + staff response/status	/feedback	Firestore	COMPLETE	Action + response workflow.
BR18	Volunteering	1	VOLUNTEERING category	/search	Strapi	COMPLETE	Visibility requirement met by content.
BR19	Alumni Engagement	1	ALUMNI category	/search	Strapi	COMPLETE	Visibility requirement met by content.
BR20	Jobs & Internships	3	JOB category + dashboard highlight	/	/search	COMPLETE	Targeting supported.
BR21	Facility Issue Reporting	2	Typed issue report + staff workflow	/facility-issues	Firestore	COMPLETE	Administrative status workflow.
BR22	Staff Directory	2	STAFF_DIRECTORY category	/search	Strapi	COMPLETE	Searchable directory content.
BR23	Financial Support	3	FINANCIAL_SUPPORT category	/search	Strapi	COMPLETE	Information requirement met; finance-specific transactional

ID	Requirement	Official marks	Implementation	UI	Data	Status	Evidence / limitation
							action not separately built.
BR24	Sports & Recreation	2	SPORTS category	/search	Strapi	PARTIAL	Visibility exists; dedicated sports-facility booking workflow not separately implemented.
BR25	Dining Information	1	DINING category	/search	Strapi	COMPLETE	Menu/operating-hours information.
BR26	Printing Services	1	PRINTING category	/search	Strapi	COMPLETE	Service information.
BR27	Textbook Exchange	1	TEXTBOOK listings	/search	Strapi	PARTIAL	Listings are visible/searchable; students cannot self-list through a marketplace workflow.
BR28	Guest Lectures	1	GUEST_LECTURE category	/search	Strapi	COMPLETE	Distinct information category.
BR29	Wellbeing Support	3	WELLBEING category	/search	Strapi	COMPLETE	Counselling/wellbeing information.
BR30	IT Support Info	2	IT_SUPPORT category	/search	Strapi	COMPLETE	Technical support information.
BR31	Library Resources	2	LIBRARY category	/search	Strapi	COMPLETE	Resources/hours/borrowing information.
BR32	Student Life Highlights	1	STUDENT_LIFE category	/search	Strapi	COMPLETE	Past events/achievement visibility.
BR33	AI Assistant	9	Retrieval + grounded Gemini integration + deterministic fallback	/ai	src/lib/ai.ts	PARTIAL	Gemini model call not executed in tested environment; fallback is functional and tested.

16. 100-Mark Judging Alignment

The official judging criteria divide evaluation into Code Review (60) and Product Pitch (40). The table below is an evidence-coverage plan, not a predicted score. [filecite?turn36file6](#)

Marks	Criterion	What judges need	Current evidence	Gap / preparation
45	Requirement Coverage & Depth	BR1–BR33 traceability; high-value workflows; reusable architecture; NFR evidence	Detailed requirement rows; high-depth BR8/BR12; BR33 limitation explicit	Reconcile requirement count; do not overclaim AI.
9	Code Quality & Best Practices	Type safety, reusable components, no clutter, error handling, maintainability	TypeScript clean; lint exit 0; shared UI vocabulary; unused dependencies removed; reproducible E2E	Two lint warnings remain; vendor bundle remains large.
6	Report Quality	Design + architecture + testing + strong evidence	Design reasoning, architecture, test matrix, traceability, limitations, evidence index	Keep final report and repo documentation consistent.
12	Requirement Completion, Understanding & Prioritisation	Explain what was built, left out and why	Clear Level 1/2/3 strategy; BR24/27/33 limitations explained	Prepare concise answers for every partial.
8	Functionality & Live Demo	Smooth demo matching claims	Committed 12-step E2E golden path; 0 console errors	Do not demo live Gemini as AI.
8	Solution Defence	Architecture, security, trade-offs, testing	Strong evidence on RBAC, booking, content engine, AI fallback and defect fixes	Know every limitation and why it exists.
4	Innovation & Creativity	Distinctive/clever approach	Reusable 22-category engine + action workflow split + grounded assistant/fallback	Do not frame AI alone as innovation.
8	Presentation Delivery	Clear structure, timing, confidence	8-minute pitch plan aligned to problem → solution → demo → architecture → innovation → close	Stop before 8 minutes; use 6-minute warning.

17. Code Quality & Best Practices

17.1 Strengths

- TypeScript throughout the frontend and clean type checking.
- Shared UI component vocabulary reduces visual and implementation duplication.
- Thin typed client libraries separate UI concerns from Strapi/Firestore access.
- Reusable content-item architecture avoids bespoke backend/page duplication.
- Central permission mapping and Firestore Rules provide a consistent authorization model.
- Deterministic document IDs prevent duplicate event/society registrations.
- Automated tests cover core logic, authorization and integration behaviour.

17.2 Technical debt / limitations

- The Firebase vendor bundle remains large despite route-level splitting.
- No custom server layer means secret proxying and centralised server-side orchestration are not present.
- Room booking read scope is broader than ideal for signed-in users.
- A narrow concurrency window remains around booking creation/approval despite approval-time transactional re-checking.
- Formal accessibility/usability testing was not performed.

18. Git & Development Process

The rules require teams to commit every 30–60 minutes and state that sparse commit history may result in a deduction.

filecite[turn32file7]

Evidence	Observed project artifact	Interpretation
Meaningful commits	Architecture, auth/RBAC, content engine, AI/tests, documentation, E2E and performance-related commits are represented in the available history artifacts.	Progress was recorded as functional milestones rather than one final dump.
E2E commit	7e809440 — committed reproducible Playwright smoke suite and fixed emulator/profile issues.	Adds reproducibility and demonstrates testing-driven defect discovery.
Performance commit	129b8c2 — route-level code splitting and unused dependency cleanup.	Shows NFR-focused refinement rather than feature-only work.
Final verification	6e1e4e9 — fixed repeated-run E2E strict-mode issue; final gate green.	Demonstrates final hardening and verification.

Because the accessible connector did not expose the full private repository history directly, exact commit-time cadence is not independently re-derived here. The report therefore does not claim that every 30–60 minute interval was perfectly satisfied; it documents the available Git evidence and leaves exact cadence verification to the final repository state.

19. Team Contribution

The report intentionally does not invent skill profiles or detailed ownership for Helika Athukoralla or Vinuli Ranasinghe. The official rules place responsibility for the submission on the team. Contribution statements should therefore be finalised from the actual Git author history and the team's own verified record.

Member	Verified reporting position	Evidence rule
Maleesha Dewshan	Team member; detailed role ownership should be stated only where Git or team records support it.	Use commit authorship + explicit team record.
Helika Athukoralla	Team lead; Frontend implementation	Use commit authorship + explicit team record.
Vinuli Ranasinghe	Team member; Backend implementation	Use commit authorship + explicit team record.

20. Libraries, APIs & AI Disclosure

The rules permit publicly available libraries, frameworks and APIs but require disclosure. They also permit AI tools as aids provided the core solution remains the team's own work. filecite[turn32file5]

Technology	Role in UCL ONE	Disclosure
React 19 / TypeScript / Vite / Tailwind CSS	Frontend application	Public frameworks/tooling
react-router-dom / lucide-react	Routing and UI icons	Public packages
Firebase JS SDK / Firebase Admin SDK	Auth, Firestore client access and emulator seeding	Public SDKs
Strapi Community Edition / SQLite	Content management engine	Public platform/database
Vitest / Testing Library / rules-unit-testing	Automated testing	Public test tooling
Playwright	Committed E2E smoke testing	Public test tooling
Google Gemini API	Optional live AI generation path	Third-party API; live model execution not

Technology	Role in UCL ONE	Disclosure
		demonstrated
Claude (Anthropic)	Coding aid during development	AI assistance disclosed; not treated as an independent author of the solution

21. Future Development Roadmap

Horizon	Work	Reason
Immediate	Reconcile BR/NFR counts across README, report, presentation and traceability.	Remove documentation inconsistency before judging.
Immediate	Run final demo exactly against the submitted environment and capture evidence/screenshots.	Ensure the live demo matches report claims.
Short-term	Server-side Gemini proxy; repeat live AI validation.	Protect API credentials and demonstrate BR33's actual AI path.
Short-term	Complete sports booking and textbook marketplace using existing workflow patterns.	Close BR24 and BR27 without introducing unrelated architecture.
Short-term	Add cache/retry/offline handling and continue bundle profiling.	Improve NFR2/NFR3.
Production	Atomic reservation service, monitoring, redundancy, production Firebase deployment and formal usability/security testing.	Move from hackathon prototype to operational service.

22. Judge Q&A — High-Value Defence Pack

Q: Why did you choose Strapi + Firestore?

A: Because the problem contains two different data behaviours: read-heavy, staff-authored campus information and user-specific transactional workflows. Strapi provides a reusable CMS/content model; Firestore provides transactional document storage and server-side security rules.

Q: Why not build 20 separate modules?

A: The official problem allows prioritisation. A reusable content engine gives broad requirement coverage without multiplying schemas, APIs and UI patterns. Engineering time is then spent on workflows where state and authorization actually matter.

Q: How is RBAC enforced?

A: The UI reflects roles for usability, but Firestore Security Rules are the enforcement boundary. Role and department checks are tested against the live emulator.

Q: How do you prevent duplicate society/event registrations?

A: Deterministic document identifiers combine the target item and user identity, making the operation idempotent rather than creating multiple registrations.

Q: How does room conflict detection work?

A: The system compares requested time ranges using overlap logic, then the approval workflow re-checks approved bookings inside a Firestore transaction. A narrow production race window remains and is documented.

Q: Is the AI really using Gemini?

A: The Gemini integration is implemented, but the model call has not executed in this tested environment because the API key was not configured. The demonstrated assistant path is the deterministic retrieval fallback.

Q: How do you reduce unsupported AI answers?

A: The retrieval layer first selects content from UCL ONE. The Gemini prompt is grounded in that retrieved content, while the fallback only returns platform content. This reduces unsupported answers; it does not eliminate hallucination risk.

Q: What did you leave incomplete?

A: Sports booking, textbook self-listing and live Gemini execution remain incomplete/partial. These were documented rather than presented as finished.

Q: What is the biggest technical limitation?

A: For the prototype, the largest remaining gaps are live AI validation, production-scale reliability/performance evidence, and a few deliberately simplified workflows.

Q: How did testing improve the product?

A: Testing found real issues: Firestore query/rule incompatibility, AI retrieval ranking, emulator identity/profile failures, silent profile-null actions and a repeat-run E2E strict-mode issue. Each was corrected and the final gate rerun.

Q: How do you know the rules work?

A: The Firestore suite executes against the live emulator and includes role, department and privilege-escalation cases rather than only unit mocks.

Q: Why no custom backend?

A: The six-hour constraint favoured a smaller architecture. The frontend uses typed client libraries for Firestore and Strapi directly. A server layer is a clear future improvement for secret handling and centralised orchestration.

Q: How does the system scale?

A: Firestore and Strapi provide independent data services and the application uses server-side filtering. Route-level code splitting reduces initial page payload. Formal production load testing was not performed, so scalability is described conservatively.

Q: What is your strongest architectural idea?

A: The reusable content engine: one content primitive supports many categories, targeting and search, allowing breadth without bespoke infrastructure.

Q: What would you change first after the hackathon?

A: Reconcile documentation counts, then add a server-side Gemini proxy and close the remaining booking concurrency/security trade-offs.

Q: Why is BR33 partial despite having AI code?

A: The requirement asks for actual AI-based conversational assistance. Code presence alone is not sufficient; the live model path must execute. It did not in this environment.

Q: How do you protect unpublished content?

A: The public Strapi API was security-tested; the earlier draft-read exposure was fixed and the final verification script reports 5/5 checks passing.

Q: Is the system production-ready?

A: No production readiness is claimed. The prototype has strong evidence in core areas, but production deployment, redundancy, live AI, formal usability testing and some concurrency/security hardening remain.

Q: How did you handle the six-hour constraint?

A: We prioritised a working core loop, then added depth to high-value workflows, then used the reusable content engine for breadth, followed by testing and documentation.

Q: What is your evidence that the demo is stable?

A: The committed Playwright smoke suite covers a 12-step golden path and passed with zero console errors in the final test gate.

Q: What is the difference between UI RBAC and server-side RBAC?

A: UI RBAC controls what the user sees; server-side rules decide what operations are actually permitted. Only the latter is a security boundary.

Q: What does NFR5 mean in your architecture?

A: Future maintainers can extend content categories through the shared content model and extend permission scopes through central mappings instead of creating independent systems.

Q: What did you deliberately not optimise?

A: We did not introduce unnecessary microservices or speculative caching infrastructure during the hackathon. We added route-level splitting only after measuring the bundle and documenting the remaining vendor size.

Q: What does your report prove?

A: It connects official requirements to implementation, tests, security evidence, Git evidence and known limitations so judges can verify the engineering decisions.

Q: If a judge finds a discrepancy, what do you do?

A: Acknowledge it and refer to the actual code/test evidence. The report intentionally includes a count discrepancy note rather than hiding it.

23. Evidence Index

ID	Evidence	What it supports
E1	Official problem statement	S1 — BR1–BR33, NFR1–NFR6, constraints and deliverables.
E2	Judging criteria	S2 — 45/9/6 Code Review + 12/8/8/4/8 Product Pitch.
E3	Rules	S3 — six-hour build, commit cadence, disclosures, AI/public-library rules, 8-minute pitch.
E4	Repository implementation	Frontend pages, lib modules, Firestore rules, Strapi CMS and scripts described in audit artifacts.
E5	Unit tests	29 passing pure-logic tests.
E6	Firestore/rules integration	44 passing tests against the live emulator.
E7	Authentication tests	3 passing tests against the live Auth emulator.
E8	E2E	Playwright golden path: 1 suite, 12 steps, 0 console errors.
E9	Security verification	5/5 CMS security checks passing after draft-content fix.
E10	Build/type/lint	Clean TypeScript check, lint exit 0, production build success.
E11	Git evidence	7e809440 E2E; 129b8c2 code splitting/dependency cleanup; 6e1e4e9 repeat-run E2E fix.
E12	AI configuration evidence	Gemini key absent in evaluation environment; live model execution not demonstrated.

24. Final Submission Quality Audit

Area	Audit result	Action/interpretation
Requirement coverage	PASS WITH DOCUMENTATION RECONCILIATION	Detailed matrix exists for BR1–BR33; count discrepancy must be corrected.
NFR coverage	PASS	NFR1–NFR6 explicitly classified with evidence and limitations.
Architecture	PASS	Architecture, data boundaries and security boundary documented.
Testing	PASS	76 automated tests + E2E + security script documented.
Security	PASS WITH DISCLOSED TRADE-OFFS	NFR4 classified complete; roomBooking read scope and client-side Gemini key risk disclosed.
AI honesty	PASS	Live Gemini execution explicitly not claimed.
Git evidence	PASS WITH LIMITATION	Meaningful commits and final hardening evidenced; exact 30–60 minute cadence not independently reconstructed here.
Team contribution	PASS WITH VERIFICATION REQUIRED	No invented roles; final authorship/contribution wording should be aligned with actual Git history.
Third-party disclosure	PASS	Libraries/APIs and AI coding aid disclosed.
Demo alignment	PASS	E2E golden path provides reproducible evidence; demo must avoid unverified Gemini claims.
Report quality	PASS	Design, architecture, testing and evidence are integrated throughout.

25. Judge Quick Reference — One Page

PROBLEM

Campus information and services are fragmented across disconnected channels. Students need one reliable, trusted access point.

SOLUTION

UCL ONE combines a reusable 22-category content engine, transactional student services, role/department authorization,

unified search and a grounded assistant/fallback.

Area	Quick answer
Architecture	React 19 + TypeScript + Vite + Tailwind; Firebase Auth; Firestore; Strapi 5 + SQLite; Firestore Security Rules; Gemini integration + deterministic fallback.
Strongest technical decisions	Reusable content engine; server-side RBAC; department scopes; deterministic event/society IDs; booking conflict logic; grounded retrieval; layered testing.
Testing	29 unit + 44 rules/integration + 3 auth = 76 automated tests; 1 committed Playwright golden path with 12 steps and 0 console errors; 5/5 CMS security checks.
Partial areas	BR24 sports booking; BR27 student textbook self-listing; BR33 live Gemini execution not demonstrated.
NFR status	NFR4, NFR5, NFR6 complete; NFR1, NFR2, NFR3 partial.
Key limitations	Vendor bundle size; no production redundancy; narrow booking concurrency window; broader roomBooking reads; no live Gemini; no formal usability study.
Defence message	We prioritised reliable high-value workflows and used reusable architecture to expand coverage without multiplying infrastructure. We disclose what remains incomplete.

26. Conclusion

UCL ONE is best understood as an evidence-driven hackathon prototype rather than a production deployment. Its central engineering contribution is the combination of a reusable information architecture with purpose-built transactional workflows and a server-enforced permission model. This allows a large requirement surface to be addressed without creating a separate technical subsystem for every information category.

The testing record is a substantive part of the engineering story: the team did not only run happy-path tests; testing exposed authorization/query incompatibilities, retrieval-ranking errors, emulator identity problems, silent action failures and an E2E repeat-run issue. These findings were fixed and the final test gate was rerun. [filecite?turn33file6?turn36file2](#)

The submission should retain this same discipline in its final documentation and pitch. In particular, the team should reconcile the 29/33 headline count with the detailed requirement rows before submission and must not present the unexecuted Gemini path as live AI. The official judging criteria explicitly reward clear prioritisation, smooth evidence-backed demonstration and strong technical defence; honest limitation handling directly supports those expectations.

[filecite?turn36file6](#)

Appendix A — Official 100-Mark Criterion Reference

Phase	Criterion	Marks	Excellent descriptor
Code Review	Requirement Coverage & Depth	45	Nearly all requirements work, with high complexity and strong non-functional quality (41–45).
Code Review	Code Quality & Best Practices	9	Clean, consistent, well-commented code; no magic numbers or clutter (8–9).
Code Review	Report Quality	6	Report clearly documents design, architecture and testing with strong evidence (5–6).
Product Pitch	Requirement Completion, Understanding & Prioritisation	12	Clearly explains what was built, what was left out, and why, with strong justification of priorities (10–12).
Product Pitch	Functionality & Live Demo	8	Demo runs smoothly and clearly shows claimed functionality working (7–8).
Product Pitch	Solution Defence	8	Confident, thorough answers with clear reasoning and technical understanding.
Product Pitch	Innovation & Creativity	4	Genuinely distinctive or clever approach.
Product Pitch	Presentation Delivery	8	Clear, confident, well-structured delivery within time.

Appendix B — Final Demo Safety Checklist

- Start from a clean, known demo state.
- Use only verified demo credentials and seeded content.
- Demonstrate student login → dashboard → targeted content → search → event interest → society sign-up → room booking → lost & found → assistant fallback → logout → staff approval.
- Do not claim the assistant response is generated by Gemini unless a live model call is actually observed.
- If a feature fails, state the limitation rather than improvising a claim.
- Keep the presentation below the 8-minute hard limit; the official rules signal at 6 minutes and 8 minutes. [?filecite?turn32file5?](#)

Appendix C — Submission Consistency Checklist

- README requirement count matches report traceability.
- Presentation requirement count matches report traceability.
- BR24/BR27/BR33 wording is identical across README, report, slides and Q&A.
- NFR1–NFR6 statuses match everywhere.
- Test totals match the final test-gate output.
- AI disclosure is explicit and consistent.
- Third-party libraries/APIs are disclosed.
- No screenshot shows a feature the final repository cannot reproduce.
- No secrets are committed.
- Final GitHub branch/repository is the exact submission target.